Prompt for Incompatibility Analysis

You are an expert Python developer with expertise in control-flow analysis. You will be provided with a Control Flow Graph (CFG) and identified source-sink paths of two Python API versions (API_b and API_a). Your task is to traverse the CFG paths and analyze the incompatibilities between the two versions. Follow the detection logic step-by-step:

1. Parameter Incompatibilities

Group the source–sink paths by parameter (source) name in the form <source, <source–sink path>>. Then perform: *(1) Default Parameter Value Change Analysis (CI4).* Compare the parameter (source) names between API_b and API_a. For parameters that exist in both versions, identify the parameter default value change (CI4) if the parameter has the same name but different default values across the two versions. *(2) Parameter Addition, Deletion and Rename Analysis (CI1, CI3).* Identify parameters that are added in API_a or deleted from API_b. For each added parameter in API_a, compare it against each deleted parameter from API_b by comparing their source–sink paths: If the source–sink paths are semantically similar, identify the change as a Parameter Rename (CI3); Otherwise and the added parameter is not an optional parameter, classify the change as a Parameter Addition (CI1). The source-sink paths are semantically similar if the statements within the paths exhibit semantic equivalence. The same applies to API_b. *(3) Kwargs Key Addition, Deletion Analysis (CI2). E*xtract the keys accessed along their source–sink paths in both API_b and API_a (e.g., via kwargs.get(key) or kwargs[key]). If the accessed keys differ between the two versions, identify it as a Kwargs Key Addition/Deletion (CI2).

2. Exceptions Incompatibilities

Select the source–sink paths whose sink_type is raise, and group them by exception (sink) name in the form <sink, <source–sink path>>. Then perform: *(1) Exception Addition or Deletion (CI6).* Compare the exception (sink) names between API_b and API_a. If an exception name appears in one version but has no corresponding exception in the other version, identify the change as an Exception Change (CI6) (addition/deletion). *(2) Parameter Boundary Range Change (CI5).* For exception names that appear in both API_b and API_a, compare their corresponding source–sink paths. Extract the control-flow conditions under which the exception is raised (e.g., conditional expressions guarding the raise statement) , and synthesize the input-parameter (source) value ranges required to trigger each exception. If the exception is raised under different parameter value constraints between the two versions, report the change as a Parameter Range Change for Exception (CI5).

3. Non-local Variable Incompatibilities

Select the source–sink paths whose sink_type is global_var or attri_var, and group them by variable (sink) name in the form <sink, <source–sink paths>>. Then, for variables that appear in both API_b and API_a, compare their source–sink paths. Extract the control-flow conditions under which the variable is assigned (e.g., guarding predicates or value constraints). If the global variable is assigned under different conditions, report the change as a Global Variable Value Change (CI8). If the attribute variable is assigned in one version but unassigned in the other, report the change as an Instance Attribute Unitialized (CI9).

4. Return Incompatibilities

Select the source–sink paths whose sink_type is exit_pred. First, compare the number of predecessors (i.e., <predecessor, <source–sink path>> pairs) between API_b and API_a. Then perform predecessor mapping across versions. For each <predecessor, <source–sink path>> pair in API_b and API_a, identify corresponding predecessors across the two versions by matching them based on the semantic similarity of their source–sink paths. Identify predecessors that cannot be mapped between API_b and API_a. If a new predecessor type appears in either version, report the change as a Return Type Change. For each mapped predecessor pair, compare their corresponding source–sink paths. If the return conditions, value constraints, or controlling predicates differ between the two versions, report the change as a Return Value Change (CI7).